



UNIVERSITÀ
di **VERONA**

Deep Learning

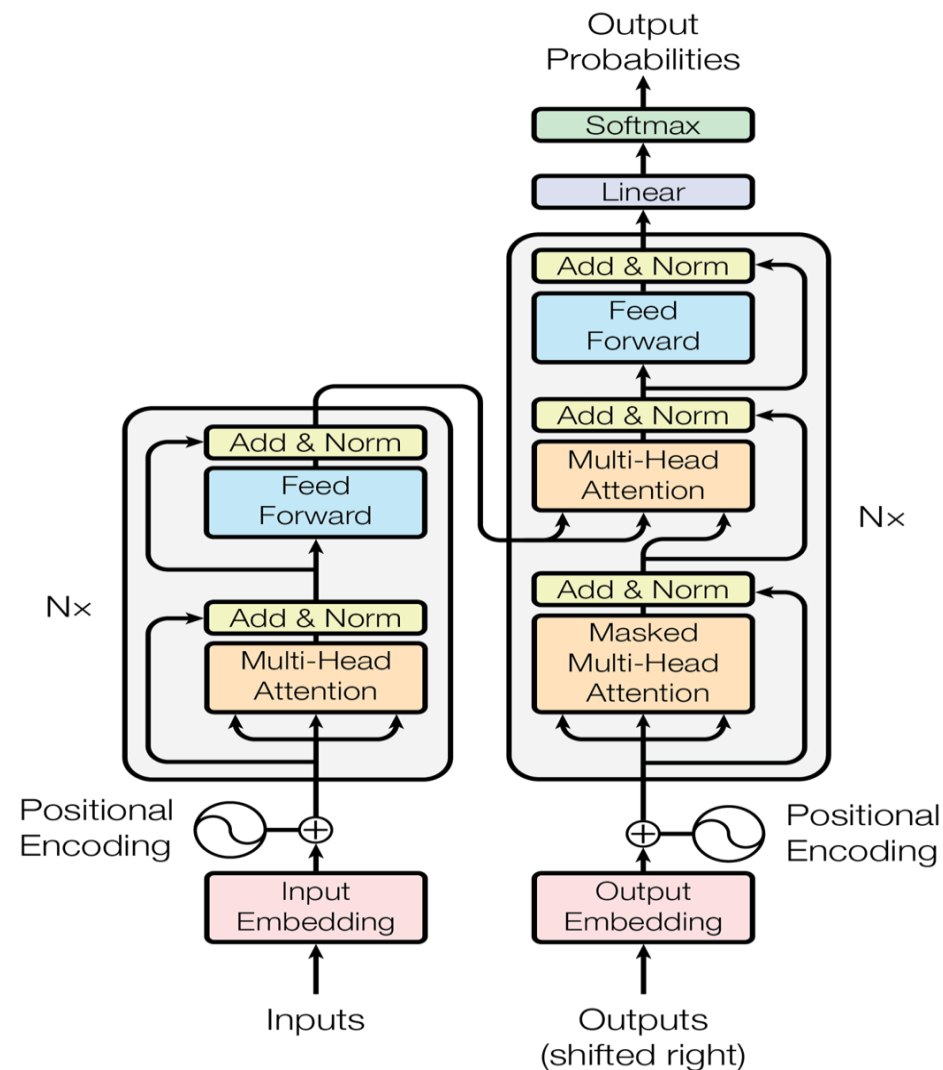
ATTENTION MECHANISM & TRANSFORMERS

Cigdem Beyan
A. Y. 2025/2026

TODAY



- Sequence-to-sequence
- (NLP) Transformers
 - Architecture
 - Attention mechanism
 - Transformer layer
 - Positional encoding...



2017



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukas Kaiser*
Google Brain
lukasz.kaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensemble, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

arXiv:2010.11929v2 [cs.CV] 3 Jun 2021

Published as a conference paper at ICLR 2021

AN IMAGE IS WORTH 16x16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy*[†], Lucas Beyer*, Alexander Kolesnikov*, Dirk Weissenborn*,
Xiaohua Zhai*, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby*[‡]

*equal technical contribution, [†]equal advising
Google Research, Brain Team
{adosovitskiy, neilhoulby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.

1 INTRODUCTION

Self-attention-based architectures, in particular Transformers [Vaswani et al., 2017], have become the model of choice in natural language processing (NLP). The dominant approach is to pre-train on a large text corpus and then fine-tune on a smaller task-specific dataset [Devlin et al., 2019]. Thanks to Transformers' computational efficiency and scalability, it has become possible to train models of unprecedented size, with over 100B parameters [Brown et al., 2020, Lepikhin et al., 2020]. With the models and datasets growing, there is still no sign of saturating performance.

In computer vision, however, convolutional architectures remain dominant [LeCun et al., 1989, Krizhevsky et al., 2012, He et al., 2016]. Inspired by NLP successes, multiple works try combining CNN-like architectures with self-attention [Wang et al., 2018, Carion et al., 2020], some replacing the convolutions entirely [Ramachandran et al., 2019, Wang et al., 2020a]. The latter models, while theoretically efficient, have not yet been scaled effectively on modern hardware accelerators due to the use of specialized attention patterns. Therefore, in large-scale image recognition, classic ResNet-like architectures are still state of the art [Mahajan et al., 2018, Xie et al., 2020, Kolesnikov et al., 2020].

Inspired by the Transformer scaling successes in NLP, we experiment with applying a standard Transformer directly to images, with the fewest possible modifications. To do so, we split an image into patches and provide the sequence of linear embeddings of these patches as an input to a Transformer. Image patches are treated the same way as tokens (words) in an NLP application. We train the model on image classification in supervised fashion.

When trained on mid-sized datasets such as ImageNet without strong regularization, these models yield modest accuracies of a few percentage points below ResNets of comparable size. This seemingly discouraging outcome may be expected: Transformers lack some of the inductive biases

[†]Fine-tuning code and pre-trained models are available at https://github.com/google-research/vision_transformer



- Transformers were originally designed to address **sequence transduction**, also known as **neural machine translation**.
 - Their purpose is to transform an **input sequence** into an **output sequence**.
- Transformers evolved from the **encoder-decoder architecture** but differ significantly.
 - Unlike Recurrent Neural Networks, Transformers eliminate **recurrence** entirely.
 - Instead, they leverage **self-attention mechanisms** for **efficiency** and **parallelization**.

ATTENTION

- Transformers analyze the **relationships between different elements** in a sequence.
- They do not process words one by one like RNNs.
- Their understanding relies almost entirely on **attention mechanisms**, particularly **self-attention**.
- **Self-attention assigns different importance (weights) to words**, helping the model grasp long-range dependencies and meaning efficiently.



Image credit: Josep Ferrer



THE SHIFT FROM RNN/LSTM TO TRANSFORMERS

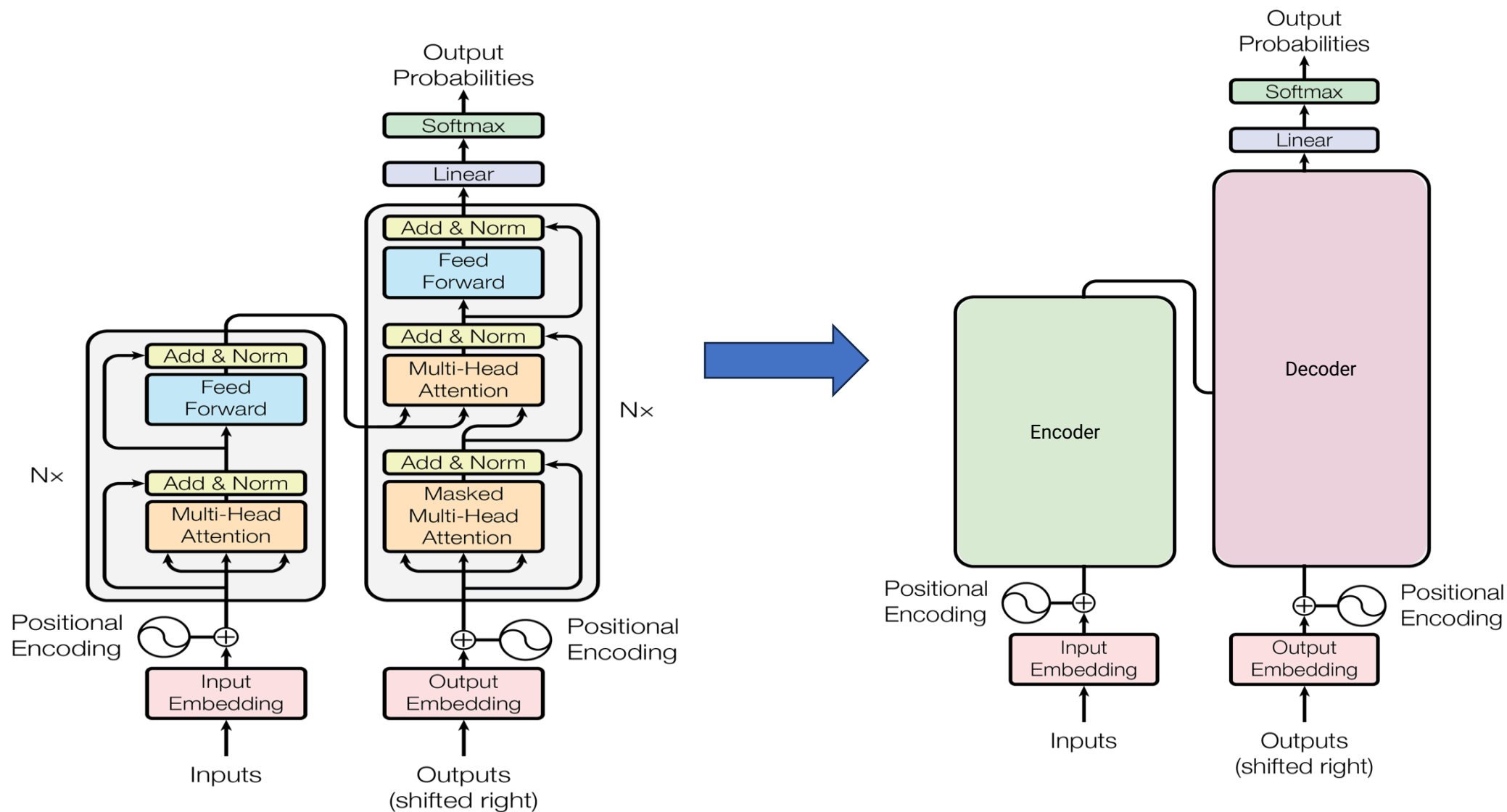
- Main problems with RNNs
 - Process the input sequentially, one after the other.
 - RNNs can't fully leverage modern GPUs designed for parallel computation
 - Struggles with **long-range** dependencies
 - Information degrades as the sequence length increases.
- Transformers
 - Attention mechanism allows focusing on important words, no matter the distance.
 - **Faster** processing with **parallel** computation i.e., more **efficient**.
 - Better **performance**.



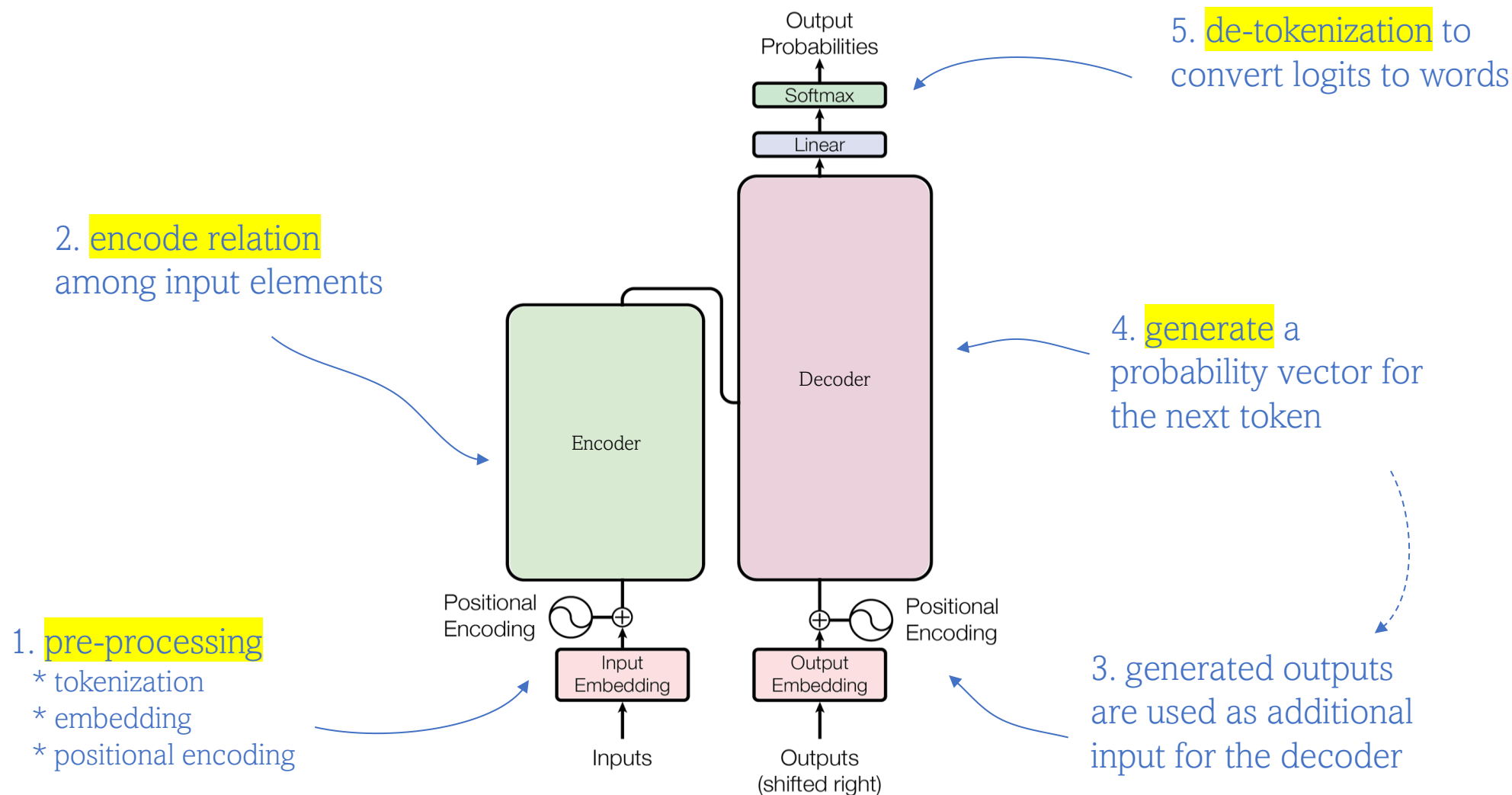
SEQUENCE TO SEQUENCE

- Before transformers, NLP (for sequence-to-sequence tasks) relied on RNN, eg. LSTM, GRU.
- Visual domain has been dominated by convolutions for decades.
- **Attention mechanism** introduced by “fast weight controllers” (Schmidhuber, 1992-1997).
- Transformer architecture stands on the shoulders of *OLD giants*.
 - Which architecture?
 - Encoder- decoder

TRANSFORMER ARCHITECTURE



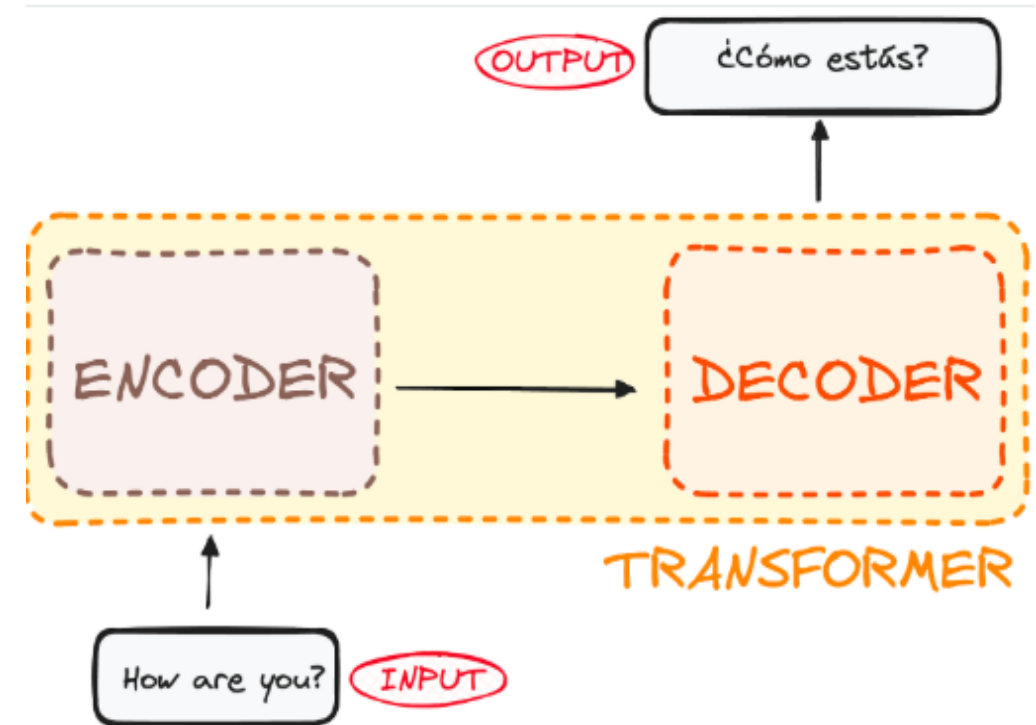
TRANSFORMER ARCHITECTURE



ENCODER - DECODER

- **Encoder:** receives entire input sequence and outputs encoded sequence of the *same length*.
- **Decoder:** predicts one word at a time, conditioned on encoder output and previously predicted words.
 - Output elements are produced one element at a time. **Auto-regressive generation**, i.e., **no parallelization**.
- E.g., The encoder takes in our input and outputs a **matrix representation** of that input: “How are you?”

The decoder takes in that encoded representation and **iteratively generates** an output: “¿Cómo estás?”



ENCODER - DECODER

- Both the encoder and the decoder are actually a stack with multiple layers (same number for each).
- All encoders present the same structure, and the input gets into each of them and is passed to the next one.
- All decoders present the same structure as well and get the input from the last encoder and the previous decoder.
- N numbers of encoder and N numbers of decoder.

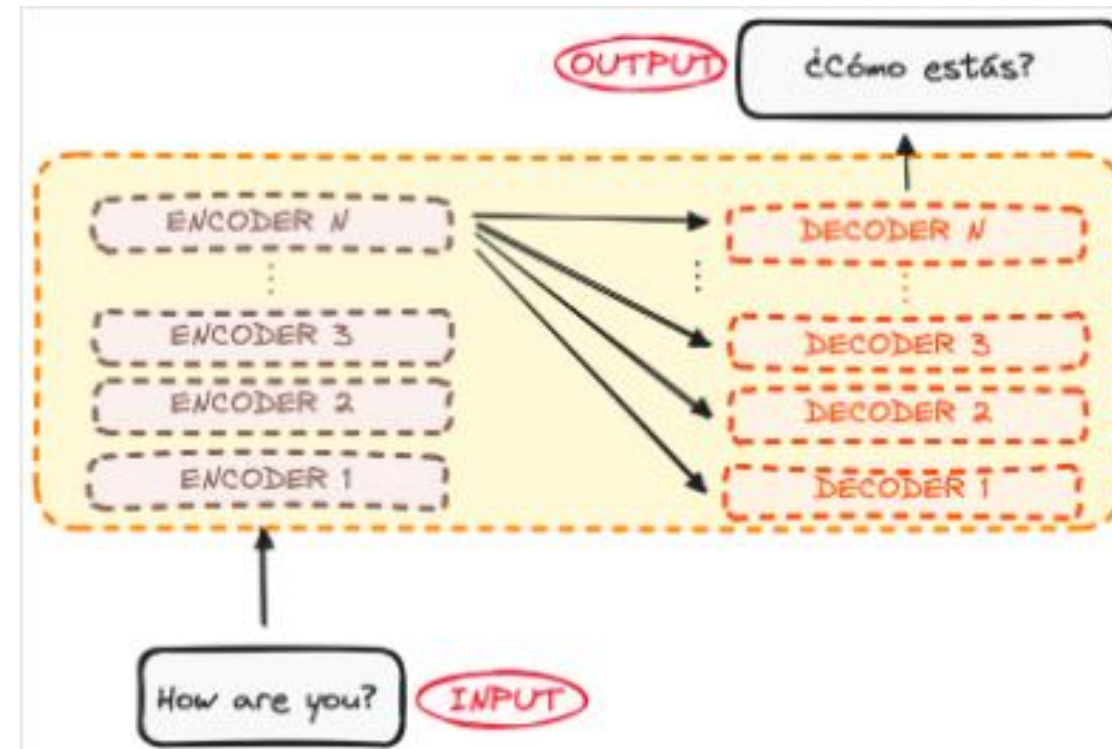
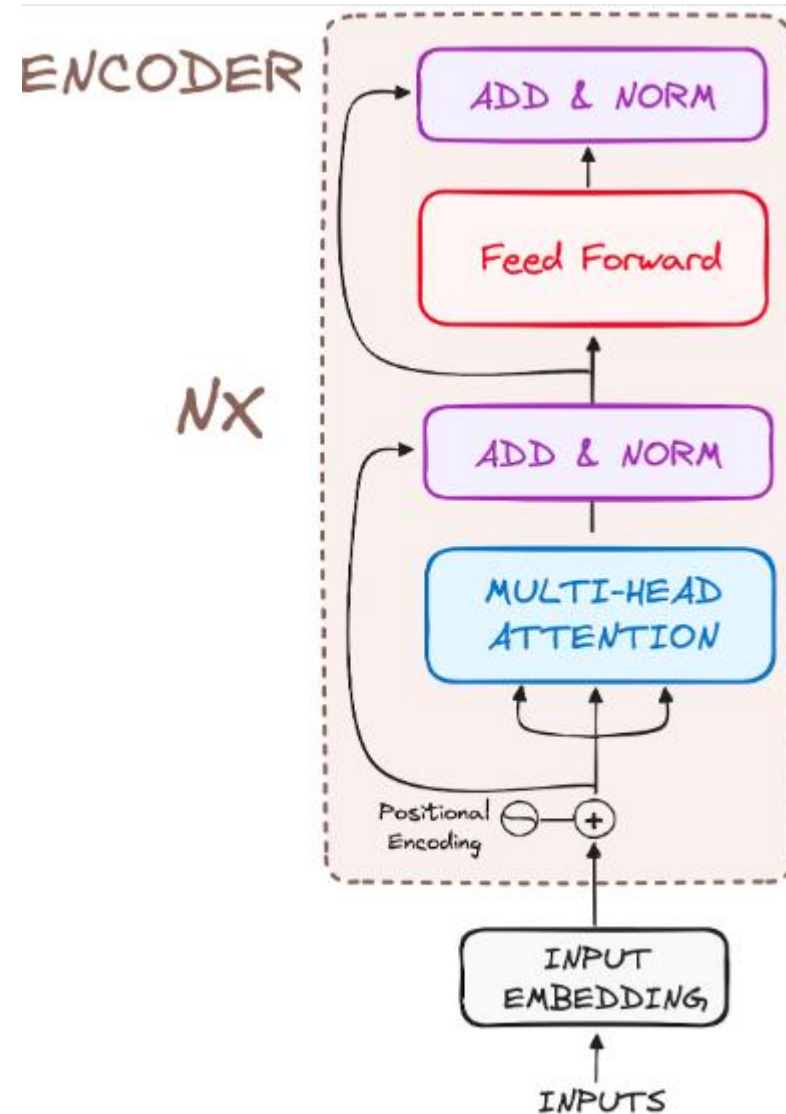


Image credit: Josep Ferrer

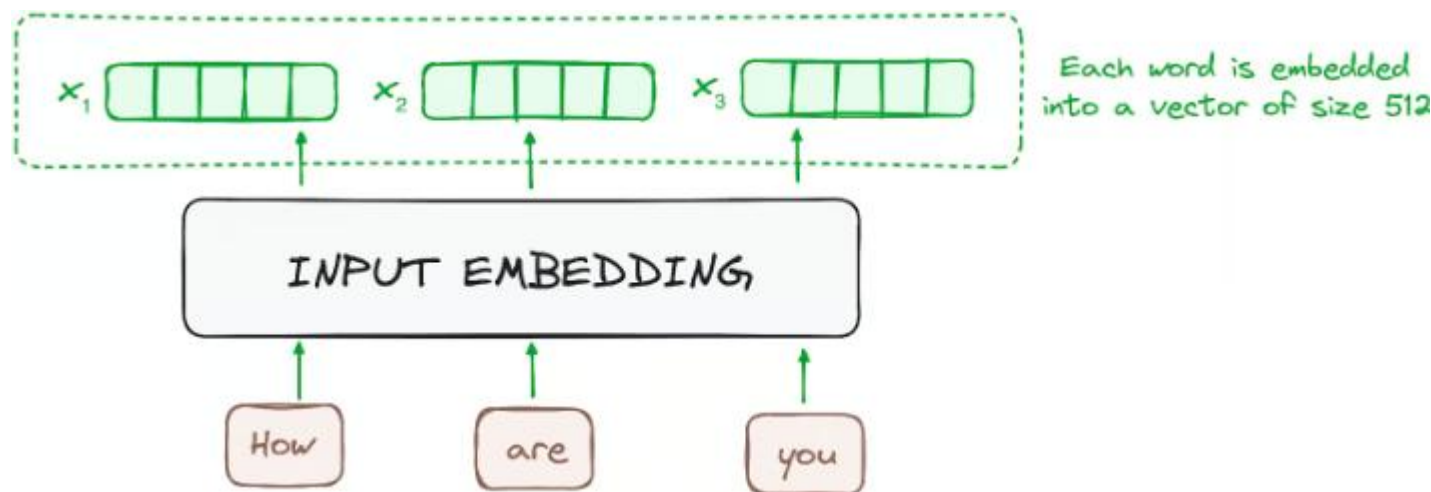
ENCODER

- The encoder transforms the input tokens into **contextualized representations**.
- It consists of:
 - Input embeddings
 - Positional Encoding
 - Stack of Encoder Layers
 - Multi-Headed Self-Attention Mechanism
 - Normalization and Residual Connections
 - Feed-Forward Neural Network
 - Output



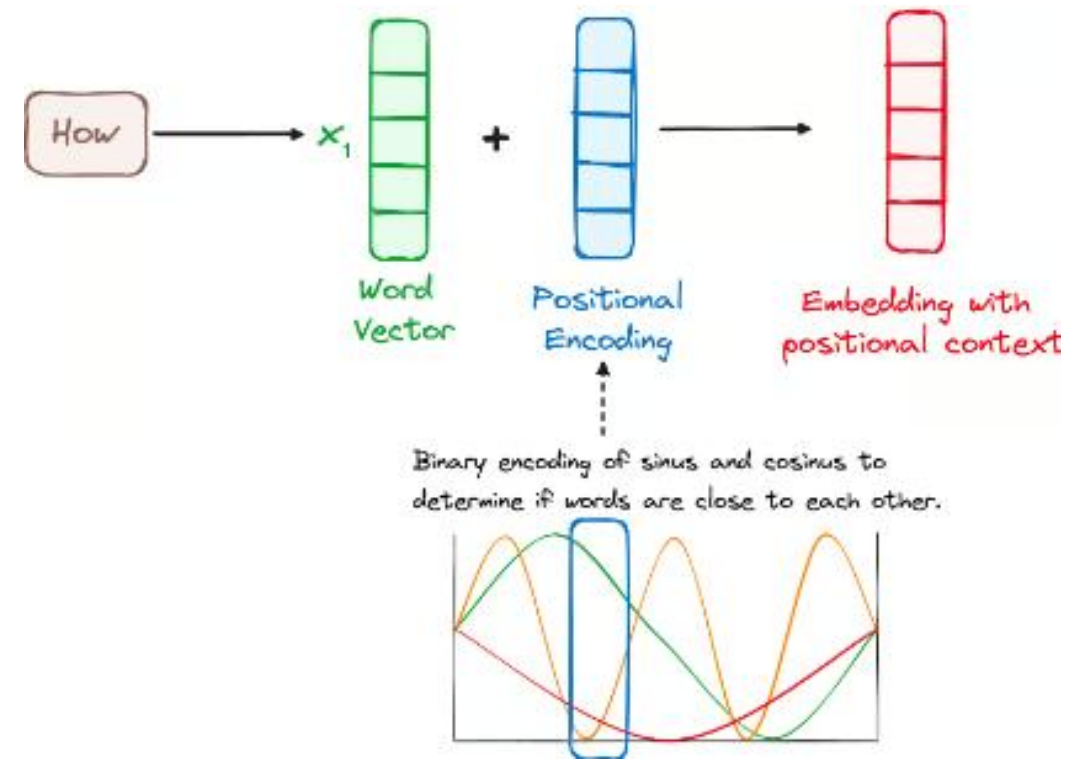
ENCODER- INPUT EMBEDDINGS

- The embedding only happens in the bottom-most encoder.
- Input tokens (e.g., words) are converted into **numerical vectors** using an **embedding algorithm**.
 - The embedding layers capture the **semantic meaning**.
- The word in each position flows through its path in the encoder.
 - All the encoders receive a list of vectors, each of fixed-sized (e.g., 512).
 - In the bottom encoder, that would be the word embeddings, but in other encoders, it would be the output of the encoder that is directly below them.



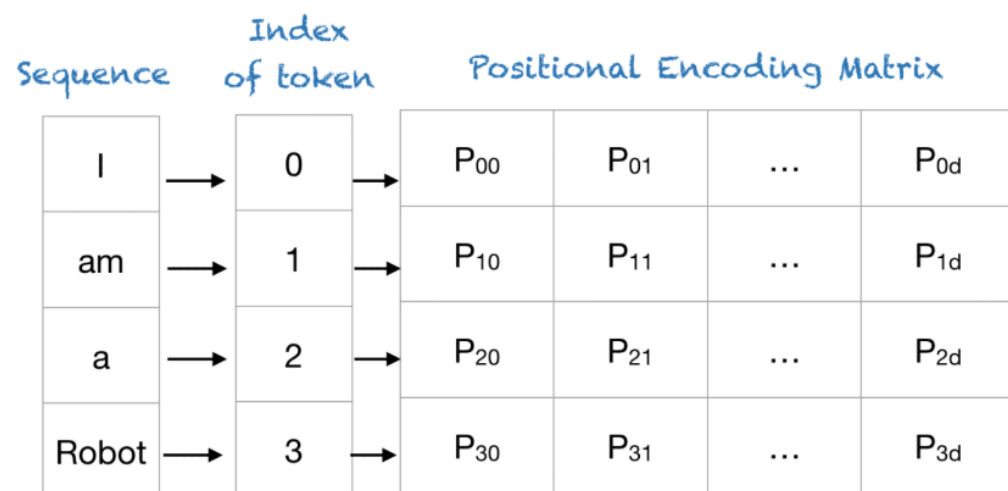
ENCODER- POSITIONAL ENCODING

- The model itself does not have any sense of **position/order** for each word.
- Positional information is provided through an **additive positional encoding** of the same dimension as the internal representation.
- With positional encoding, words will be closer to each other based on the **similarity of their meaning** and **their relative position in the sentence**, in the d-dimensional space.
- A combination of various **sine** and **cosine** functions are used to create positional vectors, enabling the use of this positional encoder for sentences of any length.



ENCODER- POSITIONAL ENCODING

- Each dimension is represented by **unique frequencies** and **offsets** of the sine/cosine wave, with the values ranging from -1 to 1, effectively representing each position.
- The output of the positional encoding layer is a matrix, where each row of the matrix represents an encoded word summed with its positional information.



Positional Encoding Matrix for the sequence 'I am a robot'

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right)$$

$$P(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

k : Position of an object in the input sequence, $0 \leq k < L/2$

d : Dimension of the output embedding space

$P(k, j)$: Position function for mapping a position k in the input sequence to index (k, j) of the positional matrix

n : User-defined scalar, set to 10,000 by the authors of [Attention Is All You Need](#).

i : Used for mapping to column indices $0 \leq i < d/2$, with a single value of i maps to both sine and cosine functions

ENCODER- POSITIONAL ENCODING

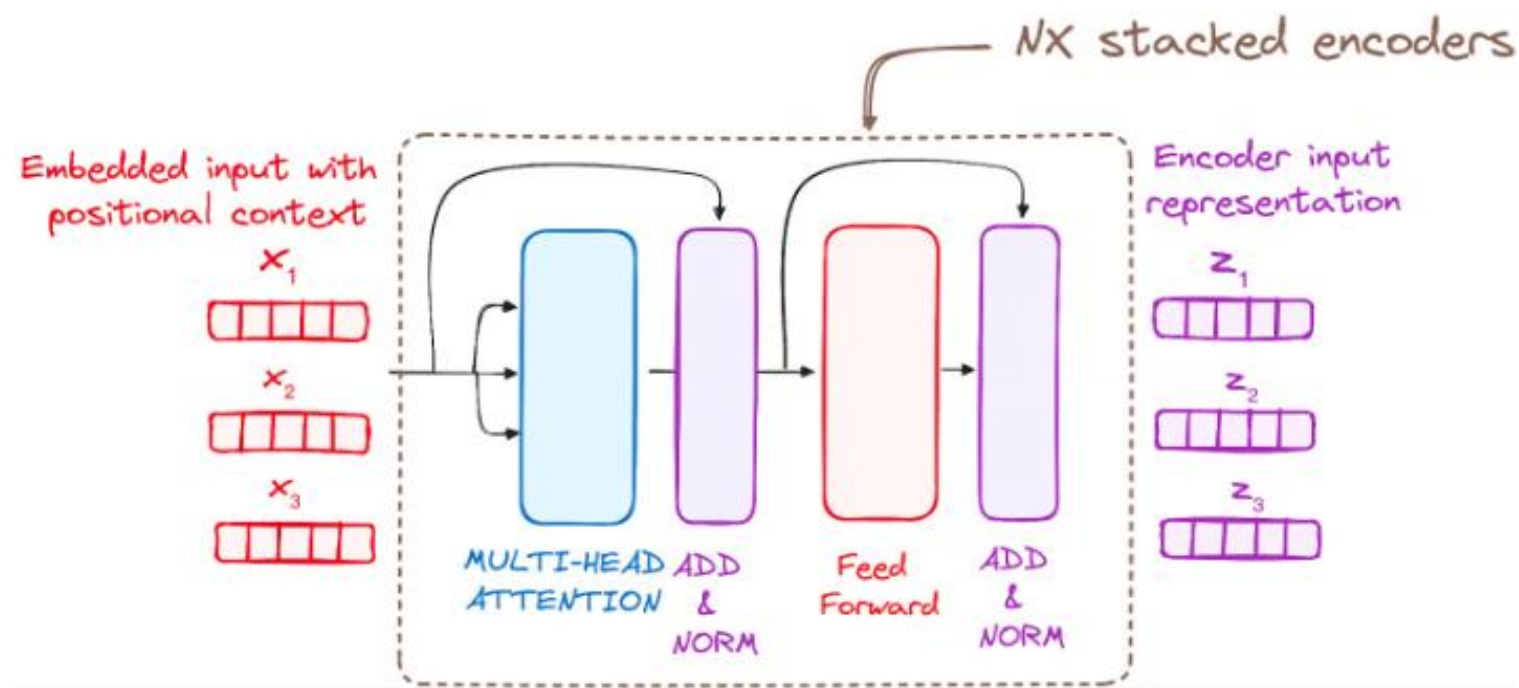
Positional Encoding
Matrix with $d=4$, $n=100$

Sequence	Index of token, k	$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

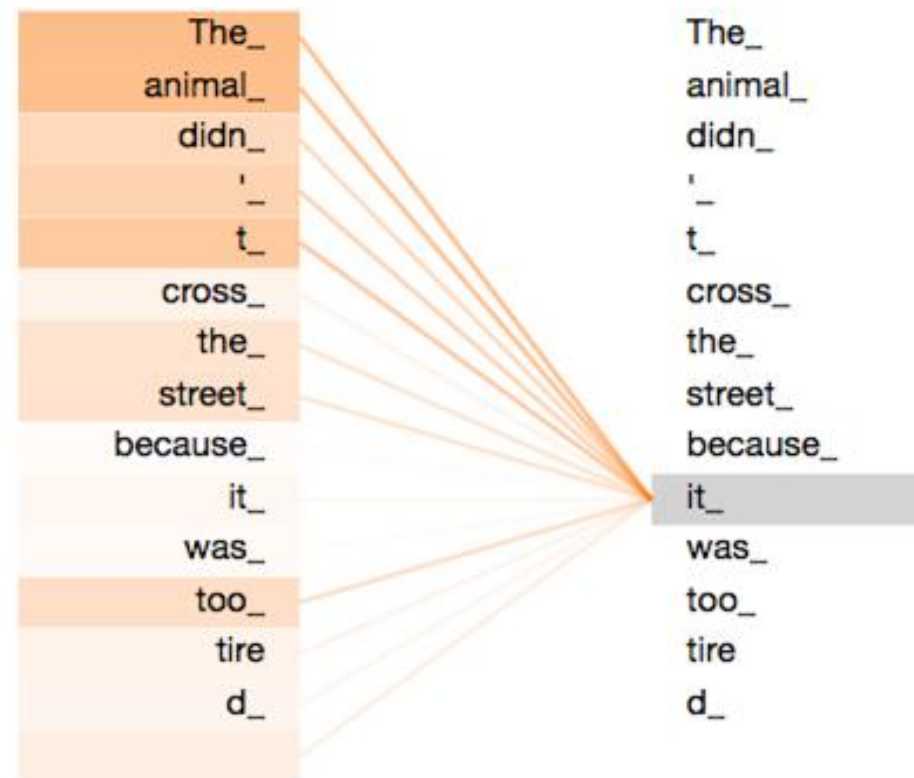
ENCODER- STACK OF ENCODER LAYERS

- An encoder layer comprises two sub-modules:
 - A multi-head attention mechanism.
 - A fully connected network.
- It also incorporates residual connections around each sublayer, which are then followed by layer normalization.



MULTI-HEAD SELF-ATTENTION MECHANISM

- Multi-head attention utilizes a specialized attention mechanism known as self-attention.
- Self-attention relates each word in the input with other words.
 - Allows the encoder to focus on different parts of the input sequence as it processes each token.
 - In other words, self-attention captures the context of other relevant words for the one we are currently processing.



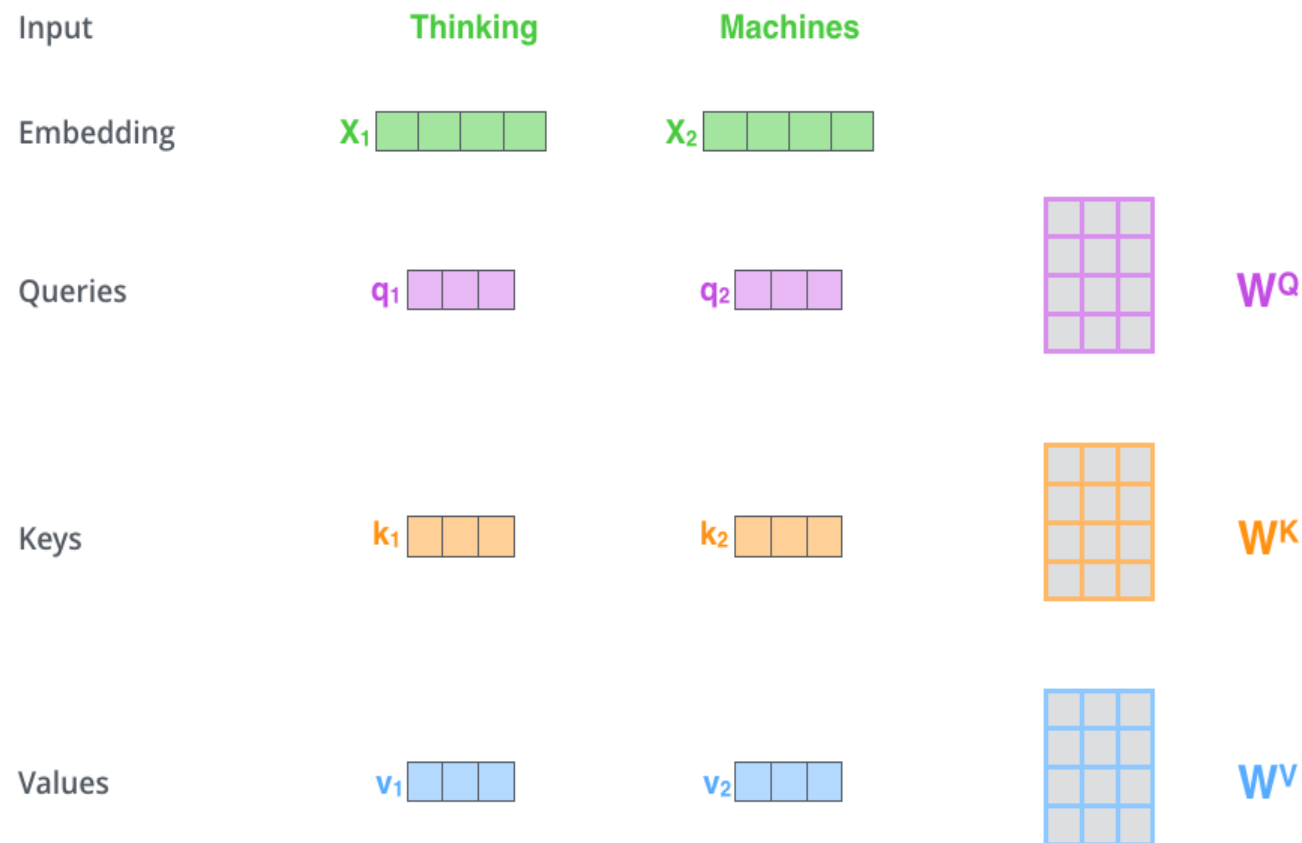
What does “it” refer to? Animal? Street?

SELF-ATTENTION MECHANISM

- Self-Attention is a mechanism for creating **arbitrarily long-range weighted dependencies** between word tokens in language models.
- How to compute self-attention?
 - Create 3 vectors from embeddings
 - **Query**: represents a specific word or token from the input sequence in the attention mechanism.
 - **Key**: the vector in the attention mechanism, corresponding to each word or token in the input sequence.
 - **Value**: is associated with a key and is used to construct the output of the attention layer.
 - When a query and a key *match* well, which basically means that they have a *high attention score*, the corresponding value is emphasized in the output.

SELF-ATTENTION MECHANISM

- **1st step:** for each word, we create a Query vector, a Key vector, and a Value vector.
- How? By multiplying the embedding by three trainable matrices (W^Q , W^K , W^V) that we learn during the training process.
- E.g., Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word.



$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

SELF-ATTENTION MECHANISM

- 2nd step: Calculate a score.
 - The score determines how much focus to place on other parts of the input sentence as we encode a word at a certain position.
- Take the dot product of the **query vector** with the **key vector** of the respective word we're scoring. So, if we're processing the self-attention for the word in position #1,
 - the first score would be the dot product of q_1 and k_1 .
 - The second score would be the dot product of q_1 and k_2 .

Input

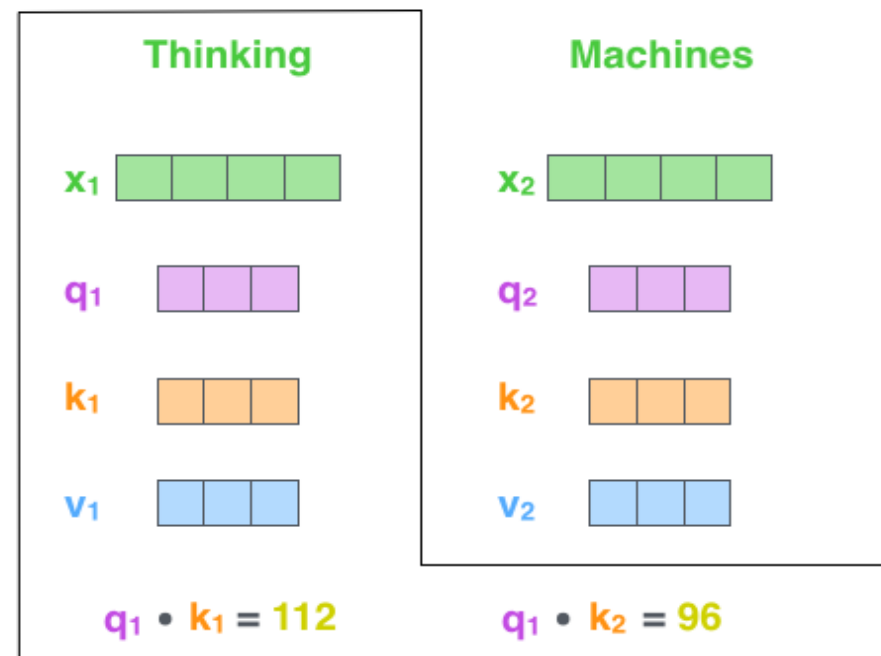
Embedding

Queries

Keys

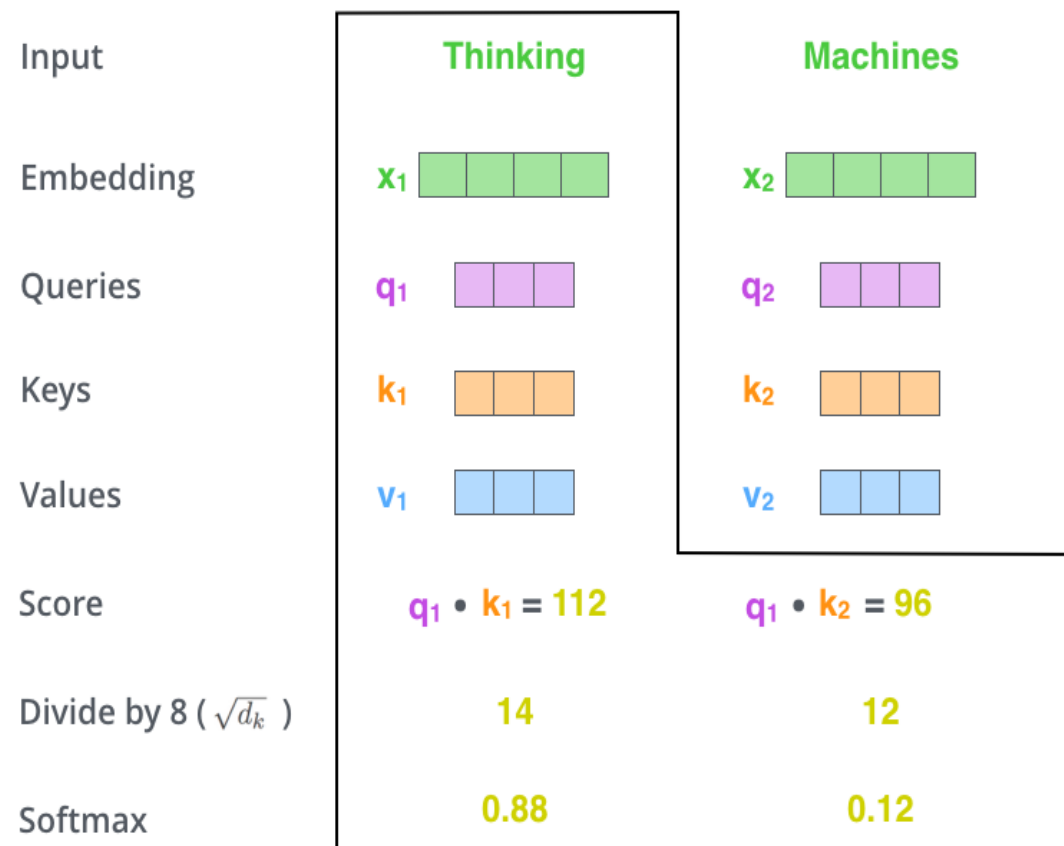
Values

Score



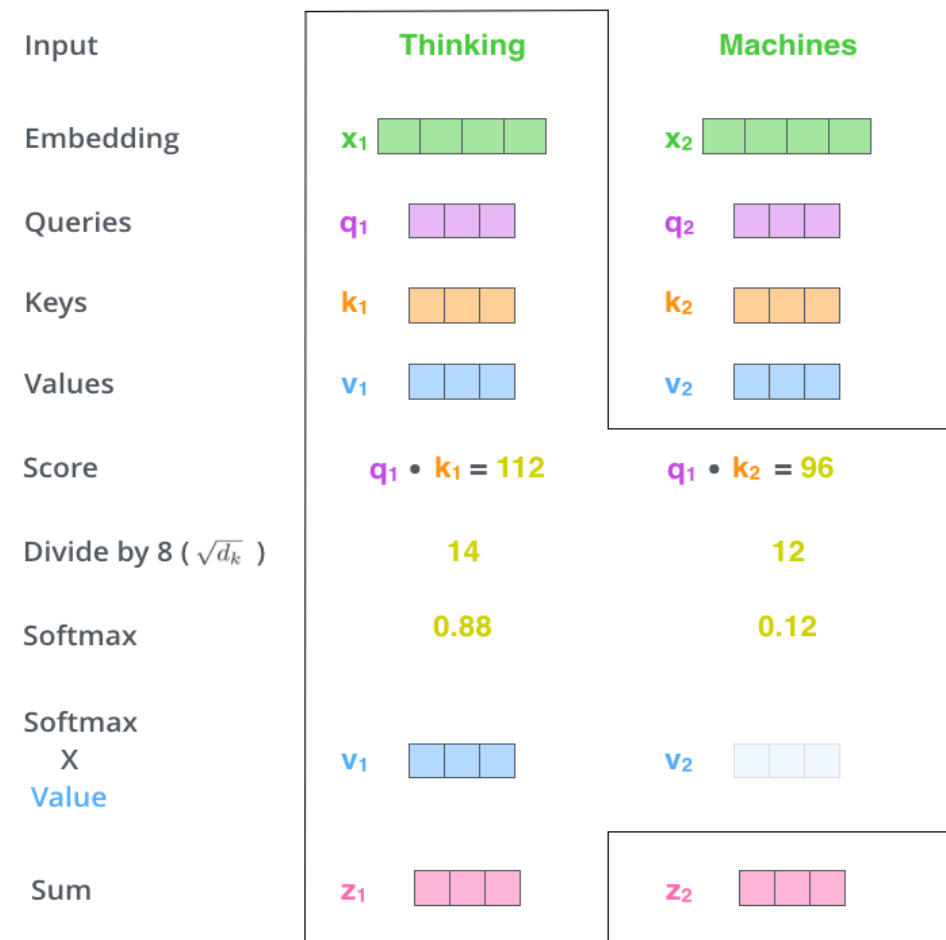
SELF-ATTENTION MECHANISM

- 3rd and 4th steps:
- The scores are **divided** by e.g., 8 (the original paper does so) to have more stable gradients.
- Then pass the result through a **softmax** operation. Softmax normalizes the scores so they're all positive and add up to 1.
- This softmax score determines how much each word will be expressed at this position.
- The word at this position will have the **highest softmax score**, but could also attend to another word that is relevant to the current word.



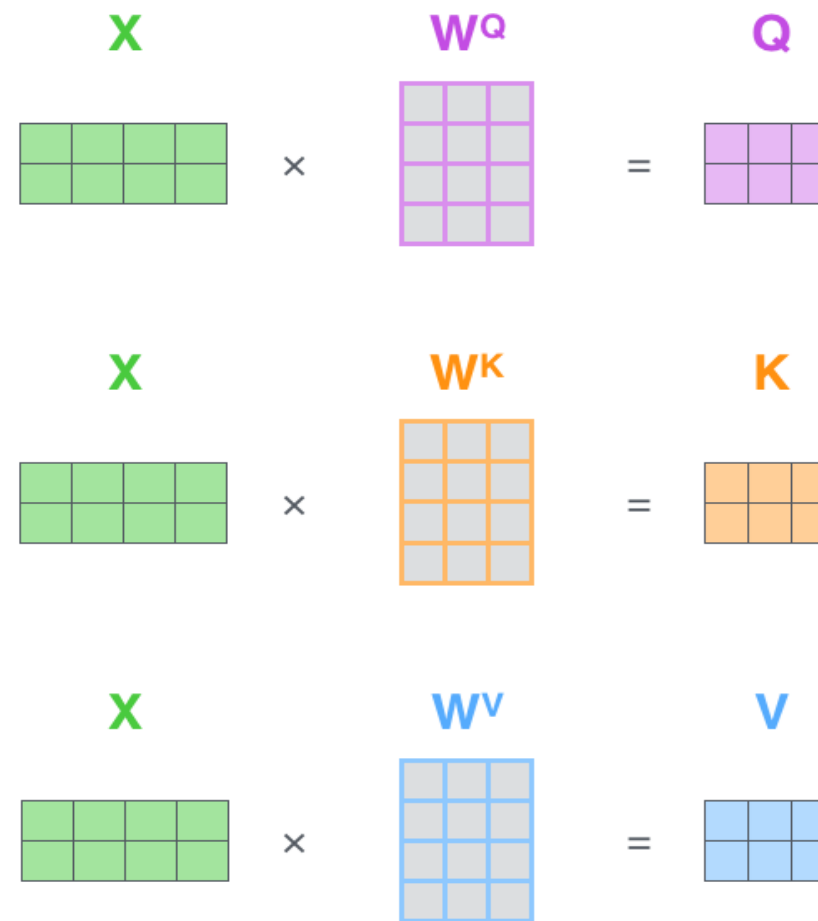
SELF-ATTENTION MECHANISM

- **5th step:** Multiply each value vector by the softmax score. This is to keep intact the values of the word(s) we want to focus on and drown out irrelevant words.
- **6th step:** Sum up the weighted value vectors. This produces the output of the self-attention layer at this position (for the first word).
- The resulting vector is one we can send along to the [feed-forward neural network](#).
- In the actual implementation, however, this calculation is done in matrix form for faster processing.



SELF-ATTENTION IN MATRIX FORM

- Every row in the X matrix corresponds to a word in the input sentence.
- The queries, keys, and values are linear functions of the inputs.
 - Calculate query, key, and value matrices by packing our embeddings into a matrix X , and multiplying it by the weight matrices we've trained (W^Q , W^K , W^V).



SELF-ATTENTION IN MATRIX FORM

- Since we are dealing with matrices, we can condense steps two through six into one formula to calculate the outputs of the self-attention layer.

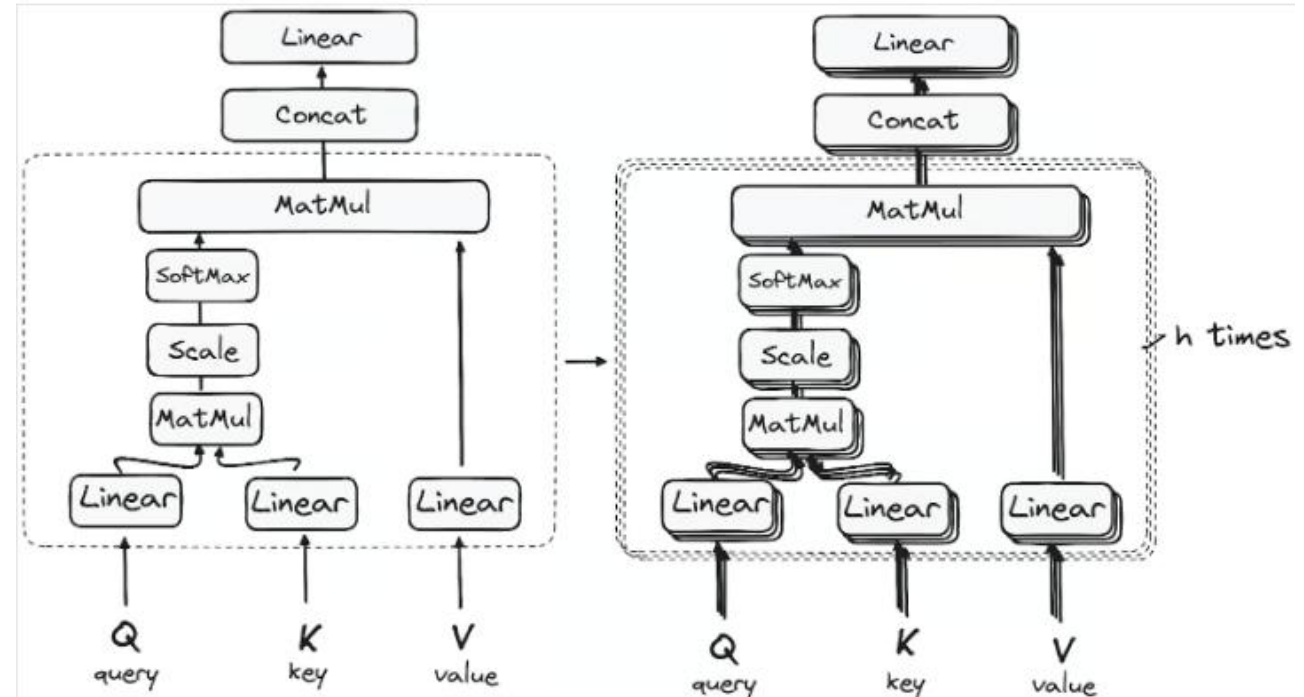
$$\text{softmax} \left(\frac{\overset{\text{Q}}{\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \end{array}} \times \overset{\text{K}^T}{\begin{array}{|c|c|} \hline \square & \square \\ \hline \end{array}}}{\sqrt{d_k}} \right) \overset{\text{V}}{\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \end{array}}$$
$$= \overset{\text{Z}}{\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \end{array}}$$

Image credit: Jay Alammar

- The dot product between query and key vectors is an unnormalized cosine similarity: indicating the alignment of two vectors.
 - If a query and key are highly aligned, **the dot product is large**, meaning the query pays more attention to that key's corresponding value.
- The dot product of Q and K form a **score matrix**, indicating how much attention each word should give to every other word.
- This matrix is later **normalized using Softmax** to get actual attention weights.
- Soft attention is differentiable, hence it is convenient to use it in NN.

MULTI-HEAD SELF-ATTENTION MECHANISM

- This first Self-Attention module enables the model to capture contextual information from the entire sequence.
- Instead of performing a single attention function, queries, keys, and values are linearly projected **h times**.
- On each of these projected versions of queries, keys, and values the attention mechanism is performed **in parallel**, yielding **h -dimensional output values**.





MULTI-HEAD SELF-ATTENTION MECHANISM

- Why multi-head attention?
 - It expands the model's ability to focus on **different positions**.
 - In our example, z_1 contains a little bit of every other encoding, but it could be dominated by the actual word itself.
 - It gives the attention layer multiple “**representation subspaces**”.
 - With multi-headed attention we have not only one but multiple sets of Query/Key/Value weight matrices.
 - Each of these sets is randomly initialized. Then, after training, each set is used to project the input embeddings (or vectors from lower encoders/decoders) into a different representation subspace.



MULTI-HEAD SELF-ATTENTION MECHANISM

- When we repeat the same self-attention calculation, multiple times, e.g., eight different times with different weight matrices, we end up with eight different Z matrices.
- Since the feed-forward layer is not expecting eight matrices, we need a way to condense these eight down into a single matrix.
 - How? We concat the matrices and then multiply them by an additional weights matrix W^O .
 - W^O is trained with the model.
- The result matrix captures information from all the attention heads.

SUMMARY



1) This is our
input sentence*

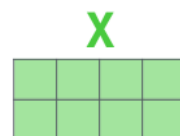
2) We embed
each word*

3) Split into 8 heads.
We multiply X or
 R with weight matrices

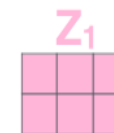
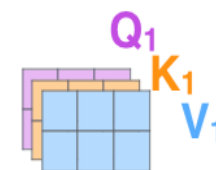
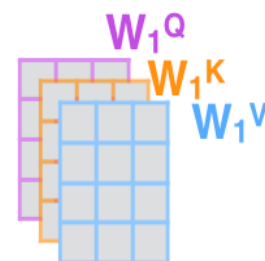
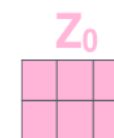
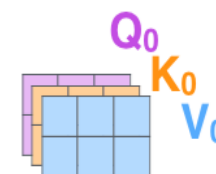
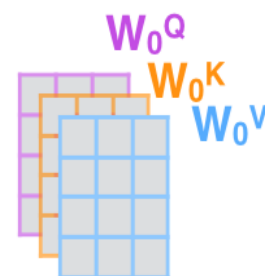
4) Calculate attention
using the resulting
 $Q/K/V$ matrices

5) Concatenate the resulting Z matrices,
then multiply with weight matrix W^O to
produce the output of the layer

Thinking
Machines



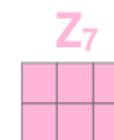
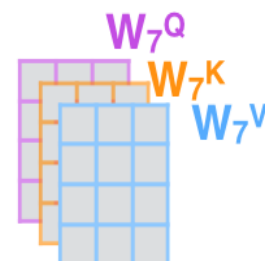
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



...

...

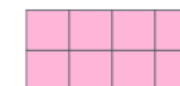
...



W^O

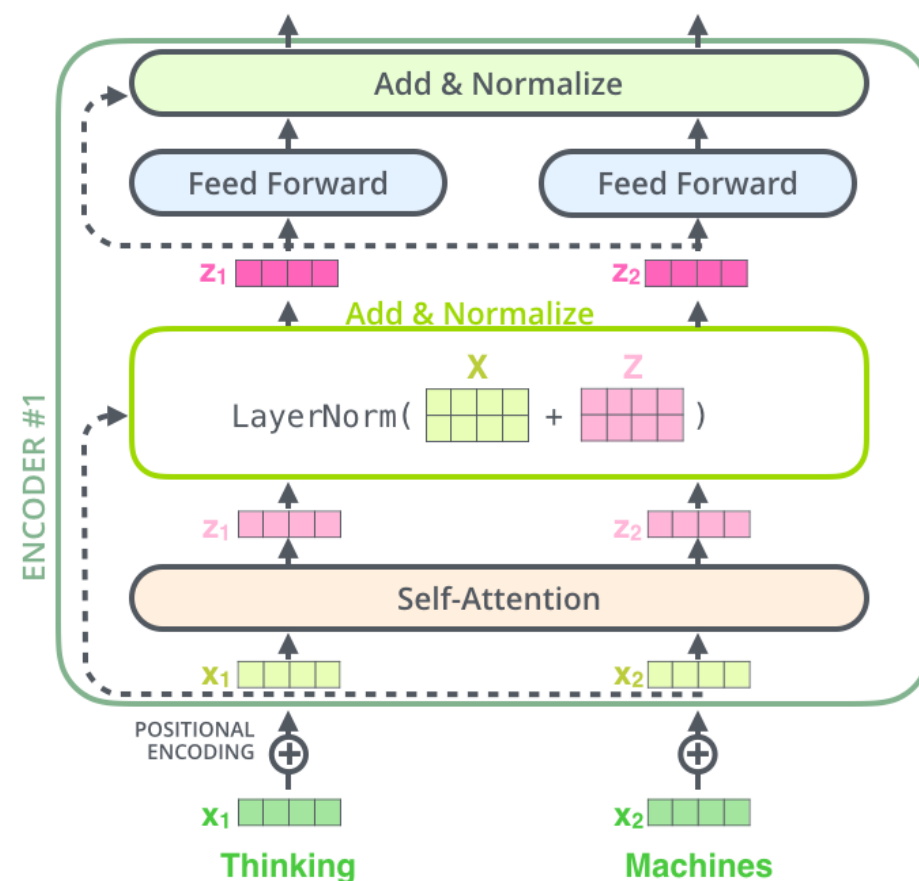


Z



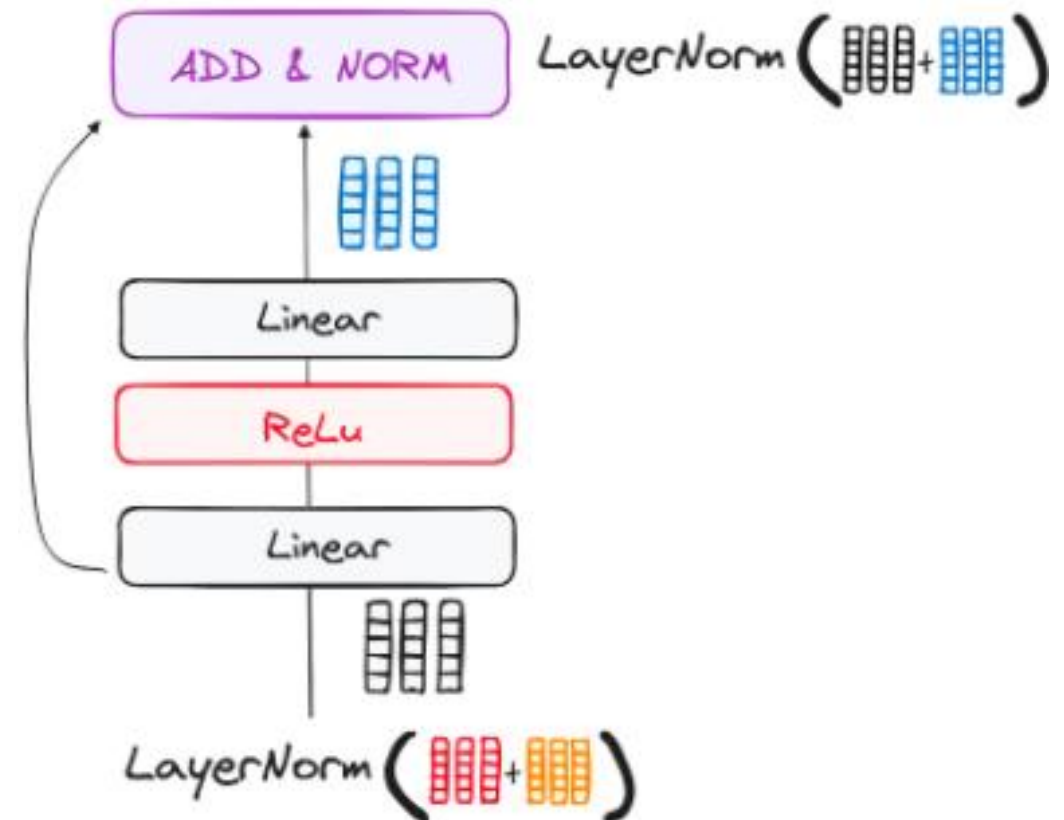
NORMALIZATION AND RESIDUAL CONNECTIONS

- Each sub-layer (SA, FF) in an encoder layer is followed by a **normalization step**.
- Also, each sub-layer output is added to its input (**residual connection**) to help mitigate the vanishing gradient problem, allowing deeper models.
- This process will be repeated after the **Feed-Forward Neural Network** too.



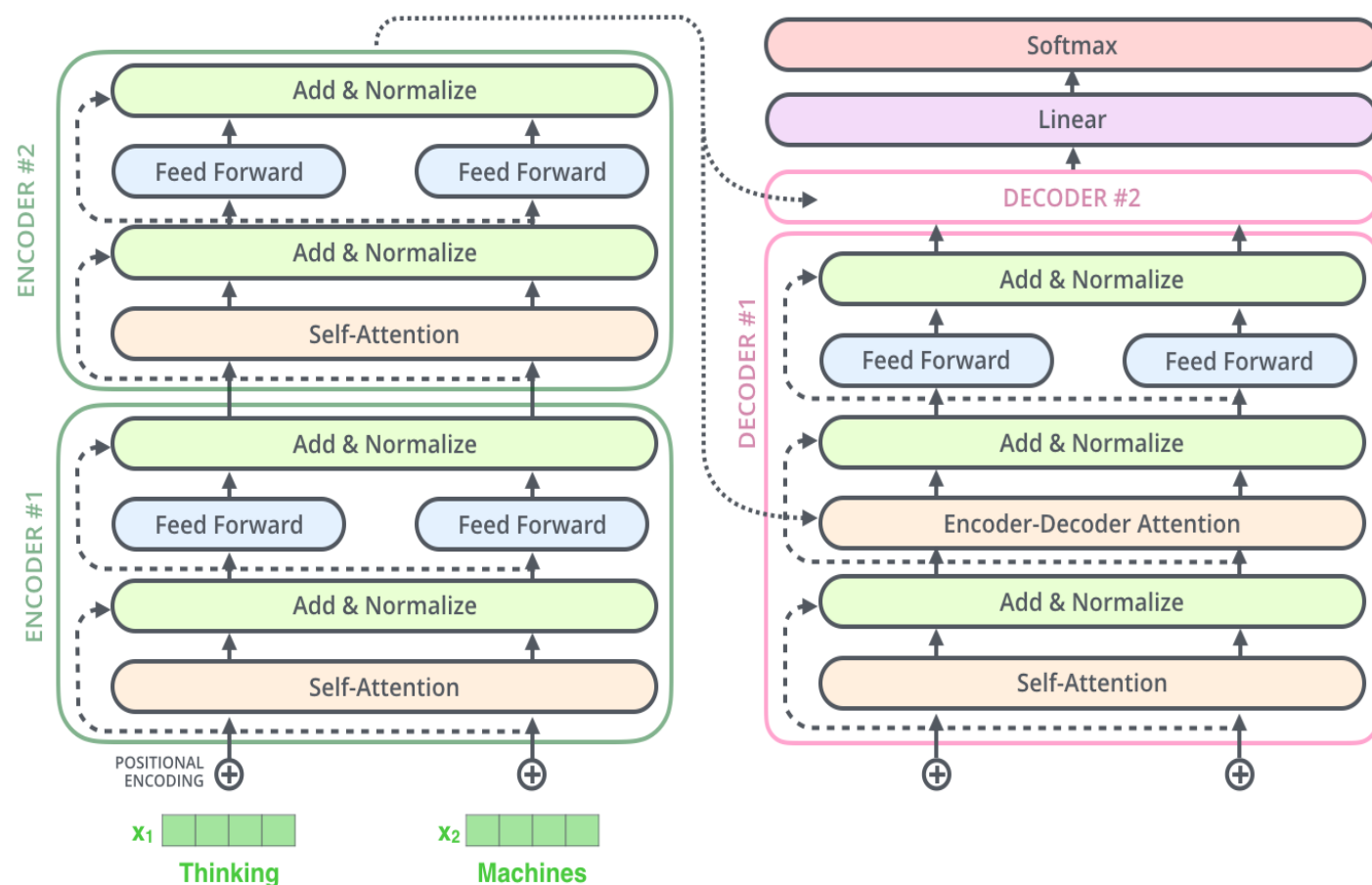
FEED-FORWARD NEURAL NETWORK

- The normalized residual output is given to a **pointwise feed-forward network**.
- This network in the original paper has **two linear layers**, with **ReLU** activation nestled between them.
- The output **merges** with the input of the pointwise feed-forward network.
- This merge is followed by another round of **normalization**.



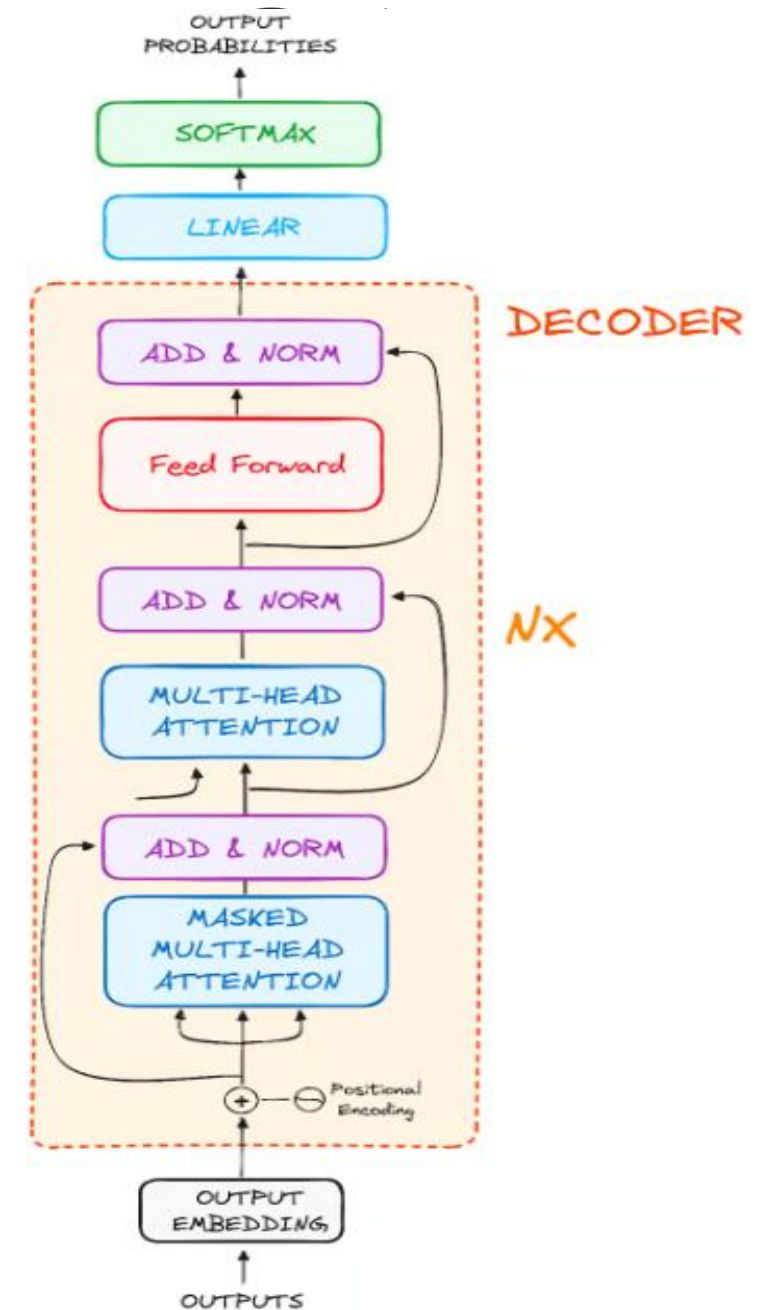
HOW ENCODERS AND DECODERS WORK TOGETHER

- The calculations in the encoder go to
 - sub-layers of the decoder, and,
 - input for the decoder
- The decoder **receives encoder outputs** and applies self-attention to its own sequence.
- The **encoder-decoder attention sub-layer** helps the decoder focus on relevant parts of the encoded input.



DECODER

- The decoder is equipped with a similar set of sub-layers.
- It has **multi-headed attention layers**, a **pointwise feed-forward layer**, and incorporates both **residual connections** and **layer normalization** after each sub-layer.
- Different from the encoder, the decoder has its “own” customized layer, called the **masked multi-head attention layer**.
 - This masking ensures that, during training and inference, the model **cannot peek at future tokens** when generating the output.
 - Masking forces the decoder to generate text **sequentially**, making it **autoregressive**.





DECODER

- The **last stage** of the decoder involves a **linear layer**, which acts as a **classifier**, followed by a **softmax function** that calculates the probabilities of different words in the vocabulary.
- Decoder is designed to generate outputs **step by step**, progressively decoding the information processed by the encoder.
 - **Work autoregressive manner**: begins with a **start token** and sequentially generates words by using its **previously generated outputs** as input.
 - At the same time, it incorporates information from the **encoder outputs**, which contain rich attention-based insights from the original input sequence.
 - This process continues **word by word** until the decoder produces a special **end token**, signaling the completion of the output.



DECODER: MASKED SELF-ATTENTION MECHANISM

- Notice that, the output of the attention module without a mask has each token attend to all other tokens. This means that all tokens have an effect on all other tokens.
- When there is **a mask**, it prevents positions from attending to subsequent positions, which means that each word in the sequence isn't influenced by future tokens.
- For instance, for the input “How are you”
 - when the attention scores for the word "are" are being computed, it's important that "are" doesn't get a peek at "you", which is a subsequent word in the sequence.
- This masking ensures that the predictions for a particular position can only depend on known outputs at positions before it.

DECODER: MASKED SELF-ATTENTION MECHANISM

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

- When applying softmax, **very large negative values** in M cause their corresponding probabilities to approach zero, effectively removing attention to future tokens.
- This ensures that when generating a word at position t , the model **cannot see** words at positions $t+1, t+2, \dots$

- Q (Query), K (Key), and V (Value) are the input matrices derived from the embeddings.
- d_k is the **dimension of the key vectors**, used for scaling.

- M is the **masking matrix**, which contains:
 - 0** for **allowed positions** (attending to past and current tokens).
 - Very large negative values (e.g., $-\infty$)** for **masked positions** (blocking attention to future tokens).

scaled scores Look-ahead mask Masked Scores

0.5	0.2	0.1
0.1	0.6	0.2
0.1	0.2	0.3

 +

0	$-\infty$	$-\infty$
0	0	$-\infty$
0	0	0

 =

0.5	$-\infty$	$-\infty$
0.1	0.6	$-\infty$
0.1	0.2	0.3



TRANSFORMER: ADVANTAGES

- Parallel computations (compared to RNN)
- Ability to handle long sentences (compared to CNN)
- **The Self-Attention Layer** computes attention with **all the words in the same sentence at once**.
 - It resorts to simple matrix computations and can be parallelized.



LANGUAGE PRE-TRAINING

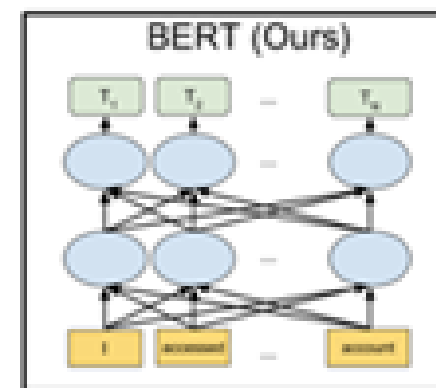
- Similar to pre-training computer vision models on ImageNet, language models can be pre-trained for tasks in natural language processing.
- Notably, the models can be pre-trained in **an unsupervised manner** from **very large datasets** and then **fine-tuned** on supervised tasks with **small datasets**.
 - There exist: a) encoder-only models, b) decoder-based models, and c) encoder-decoder models

LANGUAGE PRE-TRAINING

- Encoder models:
 - Without any task-specific head
 - Only output hidden states, which can be incorporated as features in other models.
 - They do not generate text on their own.
 - Examples of tasks: text classification, sentiment analysis, sentence similarity
 - E.g., BERT (from Google AI, SOTA for several NLP tasks), RoBERTa

Encoder only

- BERT
- RoBERTa
- Reformer
- FlauBERT
- CamemBERT
- Electra*
- MobileBERT
- Longformer

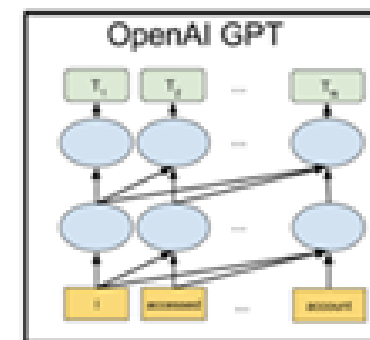


LANGUAGE PRE-TRAINING

- Decoder-based:
 - Generate text **sequentially** in an **autoregressive** fashion.
 - Tasks: machine translation, text summarization, chatbots, open-ended question-answering
 - E.g., GPT and its versions (by OpenAI, very large models, e.g., 175B parameters), Transformer XL (for longer text)

Decoder only

- Transformer-XL
- XLNet
- GPT series
- DialoGPT

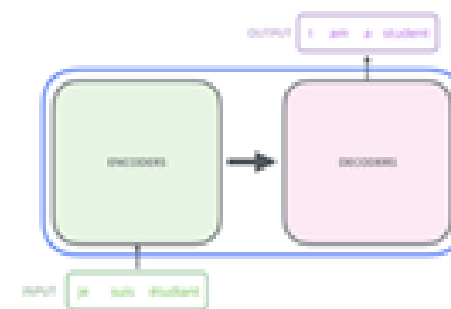


LANGUAGE PRE-TRAINING

- Encoder-Decoder Models:
 - These models combine an encoder (to process input) with a decoder (to generate output).
 - Output: A new sequence of words, making them powerful for text transformation tasks.
 - Examples of tasks: text summarization, paraphrasing, generative question answering, machine translation
 - E.g., T5, BART

Encoder + Decoder

- Transformer
- XLM
- T5
- BART
- XLM-RoBerta
- Pegasus
- mBART





FROM LANGUAGE TO VISION

- Vision Transformer (2021)
 - Object detection
 - Image segmentation
 - Image classification
 - Action recognition
 - Multi-modal tasks
 - Visual grounding,
 - Visual question answering,
 - Visual reasoning.

[cs.CV] 22 Oct 2020

Preprint. Under review.

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

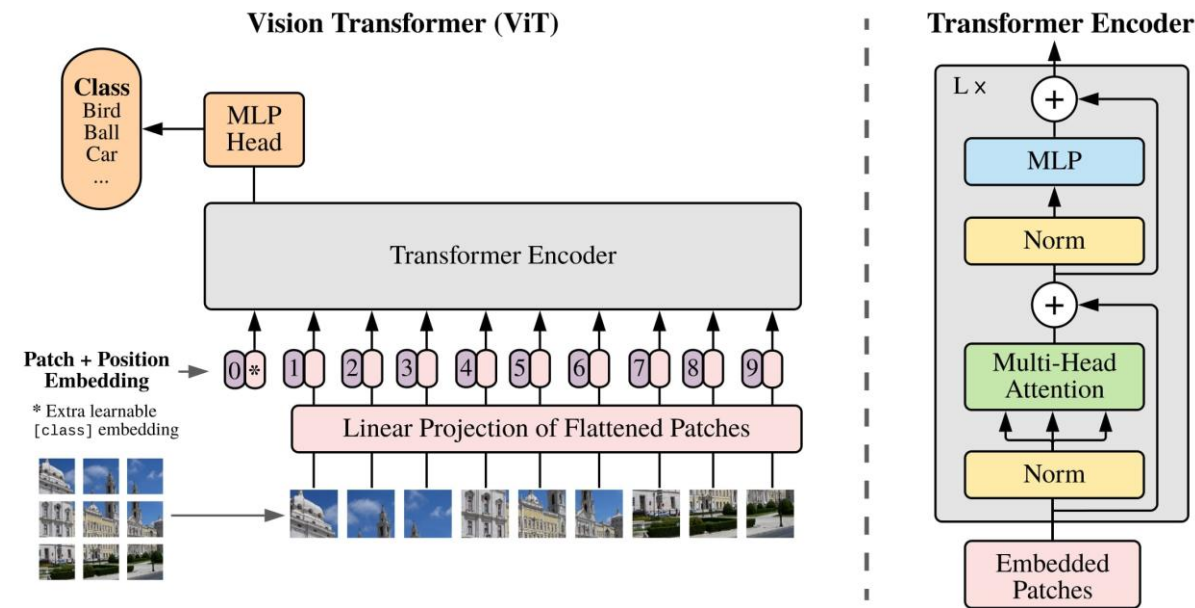
{adosovitskiy, neilhoulby}@google.com

ABSTRACT

While the Transformer architecture has become the de-facto standard for natural language processing tasks, its applications to computer vision remain limited. In vision, attention is either applied in conjunction with convolutional networks, or used to replace certain components of convolutional networks while keeping their overall structure in place. We show that this reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks. When pre-trained on large amounts of data and transferred to multiple mid-sized or small image recognition benchmarks (ImageNet, CIFAR-100, VTAB, etc.), Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring substantially fewer computational resources to train.

VISION TRANSFORMER (ViT)

- **Idea:** Apply standard transformer architecture directly to images, with the fewest possible modifications.
- **Solution:**
 - Split an image into patches
 - Flatten the patches
 - Produce lower-dimensional linear embeddings from the flattened patches
 - Add positional embeddings
 - Feed the sequence as an input to a standard transformer encoder
 - Pretrain the model with image labels (fully supervised on a huge dataset)
 - Finetune on the downstream dataset for image classification



WE COVER ViT IN THE COMPUTER VISION COURSE!!!
THIS SLIDE IS A SUMMARY AND ONLY FOR YOUR INTEREST.



That's all

Cigdem Beyan
cigdem.beyan@univr.it
<https://cbeyan.github.io/>